NimbusHost

Cloud Hosting Management System

Complete project blueprint: design, database engineering, stored programs, query optimization, audit and integrity, application, testing and documentation plan.

Course	Database Management Systems (AI203), B.Tech II, Department of Artificial Intelligence
Institute	Sardar Vallabhbhai National Institute of Technology, Surat
Database engine	MySQL 8.0+ (InnoDB)
Stack	MySQL, Node.js + Express (mysql2), React (Vite), JWT authentication
Team (2 to 4)	Member 1: _____ Member 2: _____ Member 3: _____ Member 4: _____
Roll numbers	_____

How to use this document: Sections 1 to 3 are the pre-implementation specification the guidelines ask for. Sections 4 to 10 are the database engineering work. Section 11 covers the application. Sections 12 to 16 cover documentation, testing, team plan, viva preparation and the final checklist. The scope assumes a VM-style cloud provider (like a small AWS or DigitalOcean). Infrastructure is simulated: the database models the provider control plane, and no real virtual machines are created.

Contents

1. Executive Summary **4**

1.1 Problem statement . . . 4

1.2 Core objectives . . . 4

1.3 Why this project satisfies the guidelines . . . 4

1.4 Scope boundaries . . . 4

2. Pre-implementation Specification **5**

2.1 User roles and permissions . . . 5

2.2 Key entities . . . 5

2.3 Operational workflows . . . 5

2.4 Data requirements (benchmark and demo volumes) . . . 6

2.5 Reporting requirements . . . 6

3. System Modules **7**

4. Conceptual Design: Entities and Relationships **8**

5. Relational Schema and Data Dictionary **9**

6. Database Engineering Rules **10**

6.1 Naming and type conventions . . . 10

6.2 Foreign key action policy . . . 10

6.3 Integrity constraints to implement and demonstrate . . . 10

6.4 Normalization plan and proof outline . . . 10

7. Stored Programs **12**

7.1 Stored procedures . . . 12

7.2 Stored functions . . . 12

7.3 Triggers . . . 12

7.4 Views . . . 13

7.5 Scheduled events (MySQL Event Scheduler) . . . 13

7.6 Reference code: scheduler procedure . . . 13

7.7 Reference code: audit trigger and invoice item pricing . . . 14

8. Query Optimization and Performance Engineering **15**

8.1 Method (repeat for every benchmark query) . . . 15

8.2 Benchmark queries and planned indexes . . . 15

8.3 Techniques to demonstrate . . . 15

8.4 Results table (to be filled with real numbers) . . . 16

8.5 Reference snippets . . . 16

9. Transactions, Concurrency, Audit and Integrity **17**

9.1 Concurrency demonstrations (two MySQL sessions side by side) . . . 17

9.2 Audit design . . . 17
9.3 Consistency check procedure: sp_check_consistency . . . 17
9.4 Security and access control . . . 17
10. Advanced Querying Catalogue (reports) 19
11. Application Architecture and Implementation Plan 20
11.1 Architecture . . . 20
11.2 API endpoints (REST) . . . 20
11.3 Frontend screens . . . 20
11.4 Folder structure . . . 20
12. Documentation and Deliverables Checklist 22
13. Verification: Test Plan 23
Performance and conclusion section outline . . . 23
14. Team Roles and Timeline 24
Timeline (adjust to the submission date given by the department) . . . 24
Risks and mitigations . . . 24
15. Viva Preparation: Questions to Practice 25
16. Final Submission Checklist 25

1. Executive Summary

1.1 Problem statement

Small and mid-size cloud providers must track thousands of servers, virtual machines, storage volumes, IP addresses and customers, and must bill every customer accurately for resources used by the hour. Doing this in spreadsheets or loosely structured stores leads to overbooked servers, wrong invoices, lost audit trails and slow reports. NimbusHost is a database-centred cloud hosting management system. It lets customers launch and manage virtual machines, volumes, IPs and domains, meters usage, generates invoices, enforces quotas, and gives the provider live capacity and revenue analytics.

1.2 Core objectives

1. Design a normalized (at least 3NF, targeted BCNF) schema of about 24 tables with full integrity constraints.
2. Move business logic into the database: stored procedures, functions, triggers, views and scheduled events.
3. Guarantee correctness under concurrency: no overbooked host, no double-spent credit, no duplicate IP allocation.
4. Prove performance engineering: indexes, partitioning, EXPLAIN ANALYZE before/after, repeatable benchmarks on 1 million+ rows.
5. Provide complete auditability: who changed what and when, immutable financial records, automated consistency checks.
6. Deliver a working React + Node.js application connected to MySQL with role-based access and analytical dashboards.
7. Produce the full technical report required by the course guidelines.

1.3 Why this project satisfies the guidelines

Guideline requirement	How NimbusHost satisfies it
Real-world problem, substantial database use	Cloud billing, capacity scheduling and metering are real, data-heavy problems.
Complexity and depth	24 tables, 5 user roles, 8 modules, concurrency control, partitioning, temporal pricing.
ER diagram and relational schema	Sections 4 and 5; diagram built in MySQL Workbench or draw.io.
PK, FK, domain constraints, 3NF	Section 6.3 (constraints) and Section 6.4 (normalization proof).
Multi-table JOINs, advanced topics, aggregates	Section 10: 20 reports using joins, CTEs, window functions, ROLLUP, JSON, full-text.
Functional front end with CRUD and reports	Section 11: React app for every role, 15+ screens.
Pre-implementation specification	Section 2: roles, entities, workflows, data and reporting requirements.
Test cases, results, performance analysis	Sections 8 and 13.

1.4 Scope boundaries

- **In scope:** accounts and organizations, instance catalog and pricing, hosts and regions, VM lifecycle, volumes, snapshots, IP addresses, domains and DNS records, metering, invoicing, payments (simulated gateway), support tickets, audit, analytics.
- **Out of scope:** real hypervisors, real payment gateways, real DNS servers. A simulator script generates realistic usage data instead.

2. Pre-implementation Specification

2.1 User roles and permissions

Capability	Super Admin	Support Agent	Billing Manager	Customer Owner	Customer Member
Manage regions, hosts, instance types, pricing	Yes	-	-	-	-
View all customers, instances, hosts	Yes	Yes	Yes (billing view)	-	-
Launch, stop, start own-org instances	-	-	-	Yes	Yes
Resize or terminate instances	-	-	-	Yes	-
Manage volumes, snapshots, IPs, DNS	-	-	-	Yes	Yes
View own-org invoices and usage	-	-	All orgs	Yes	Usage only
Generate invoices, apply credits, refunds	Yes	-	Yes	-	-
Pay invoices	-	-	-	Yes	-
Create tickets	-	-	-	Yes	Yes
Answer and close tickets	Yes	Yes	-	-	-
Manage org members and API keys	-	-	-	Yes	-
View audit log, run consistency checks	Yes	-	-	-	-
Revenue and capacity analytics	Yes	-	Revenue only	-	-

Enforcement happens at two levels: the API checks the JWT role, and MySQL roles with GRANT statements and tenant-scoped views restrict what each database account can touch.

2.2 Key entities

Organization (tenant), User, Region, Host, Instance Type, Image, Instance, Volume, Snapshot, IP Address, Domain, DNS Record, Pricing, Usage Record, Invoice, Invoice Item, Payment, Ticket, Ticket Message, API Key, Account Tier (quota), Audit Log, plus analytics tables (daily summary and star schema).

2.3 Operational workflows

A. Launch a virtual machine

1. Customer picks region, instance type, image, name.
2. API calls sp_launch_instance inside a transaction.
3. Quota trigger checks the organization tier limit; scheduler picks a host with enough free CPU, RAM and disk (best fit, row-locked).
4. Instance row is inserted; trigger updates host used capacity and writes the audit entry.
5. Free IP is allocated; instance status becomes running.

B. Metering to invoice to payment to suspension

1. Event ev_hourly_metering writes one usage row per running instance per hour (CPU hours, GB storage, GB egress).
2. Event ev_monthly_invoicing calls sp_run_monthly_billing, which loops over organizations with a cursor and creates invoices.
3. sp_generate_invoice prices each usage line with the pricing valid at that time (temporal lookup), adds tax, applies credit.
4. Customer pays; trigger marks the invoice paid when payments cover the total.
5. Event ev_suspend_overdue suspends instances of organizations with invoices overdue beyond the grace period; payment reactivates them.

C. Support ticket

Customer opens ticket (status open) -> agent assigned -> messages exchanged -> resolved -> closed. SLA view flags tickets not answered within the priority window.

D. Snapshot and restore

Snapshot is taken from a volume (size recorded, quota checked). Restore creates a new volume from the snapshot inside a transaction.

2.4 Data requirements (benchmark and demo volumes)

Table	Demo data	Benchmark data
regions / hosts	3 / 12	5 / 40
instance_types / images	8 / 8	8 / 12
organizations / users	20 / 40	500 / 1,500
instances	150	50,000
usage_records	20,000	1,000,000 or more
invoices / invoice_items	60 / 400	6,000 / 60,000
tickets / messages	25 / 90	3,000 / 15,000

2.5 Reporting requirements

- Monthly revenue by region, by instance type, by organization tier (with ROLLUP subtotals).
- Host capacity and utilization, hosts above 85 percent, forecast of when a region fills up.
- Top spenders, month-over-month growth, customer segmentation into quartiles.
- Overdue invoices and aging buckets (0-30, 31-60, 61-90, 90+ days).
- Per-organization cost rollup across the parent/child organization tree.
- Ticket SLA compliance and average resolution time per agent.
- Audit trail per record and per user; consistency check report.

3. System Modules

#	Module	Main tables	Key database features
1	Identity and Organizations	users, organizations, org_members, account_tiers, api_keys	Self-referencing hierarchy, quota tiers, hashed secrets, roles
2	Catalog and Pricing	instance_types, images, pricing	Temporal pricing with valid_from / valid_to, no-overlap trigger
3	Infrastructure	regions, hosts	Capacity counters, CHECK limits, capacity view
4	Compute	instances	Scheduler procedure, SKIP LOCKED, status machine, JSON config, generated column
5	Storage and Network	volumes, snapshots, ip_addresses, domains, dns_records	Unique IP allocation, cascade rules, composite unique keys
6	Metering and Billing	usage_records, invoices, invoice_items, payments	Partitioning, cursor billing, events, window-function reports
7	Support	tickets, ticket_messages	FULLTEXT search, SLA view
8	Governance and Analytics	audit_log, daily_usage_summary, fact/dim tables	Audit triggers, consistency procedure, star schema

4. Conceptual Design: Entities and Relationships

The ER diagram is drawn in MySQL Workbench (Database > Reverse Engineer after the schema is built) or draw.io, using crow-foot notation. Below are the relationships that the diagram must show.

Relationship	Cardinality	Implementation
Organization - Account Tier	Many to one	organizations.tier_id FK
Organization - Organization (parent/child)	One to many, self	organizations.parent_org_id FK to same table
User - Organization	Many to many	org_members(org_id, user_id, role) composite PK
Region - Host	One to many	hosts.region_id FK
Host - Instance	One to many	instances.host_id FK
Instance Type - Instance	One to many	instances.type_id FK
Image - Instance	One to many	instances.image_id FK
Organization - Instance	One to many	instances.org_id FK
Instance Type - Pricing	One to many (history)	pricing.type_id FK, valid_from / valid_to
Instance - Volume	One to many (optional)	volumes.instance_id nullable FK
Volume - Snapshot	One to many	snapshots.volume_id FK
Region - IP Address	One to many	ip_addresses.region_id FK
Instance - IP Address	One to one (optional)	ip_addresses.instance_id nullable FK, UNIQUE
Organization - Domain	One to many	domains.org_id FK
Domain - DNS Record	One to many	dns_records.domain_id FK
Instance - Usage Record	One to many	usage_records.instance_id (enforced by procedure, see 6.3)
Organization - Invoice	One to many	invoices.org_id FK
Invoice - Invoice Item	One to many	invoice_items.invoice_id FK
Invoice - Payment	One to many	payments.invoice_id FK
Organization - Ticket	One to many	tickets.org_id FK
Ticket - Ticket Message	One to many	ticket_messages.ticket_id FK
User - API Key	One to many	api_keys.user_id FK

5. Relational Schema and Data Dictionary

PK = primary key, FK = foreign key, UK = unique key. Types shown are the intended MySQL types. The full data dictionary in the report expands each column with constraint and description; the tables below are its core.

Table	Columns, keys and constraints
account_tiers	tier_id PK; name UK; max_instances INT; max_storage_gb INT; discount_pct DECIMAL(4,2) CHECK 0..50
organizations	org_id PK; name; tier_id FK; parent_org_id FK (self, nullable); status ENUM(active,suspended,closed); credit_balance DECIMAL(12,2) CHECK >= 0; created_at
users	user_id PK; email UK; password_hash; full_name; system_role ENUM(super_admin,support,billing,customer); status; created_at
org_members	(org_id FK, user_id FK) composite PK; member_role ENUM(owner,member); joined_at
api_keys	key_id PK; user_id FK; key_hash UK; created_at; revoked BOOL
regions	region_id PK; code UK; name; country
hosts	host_id PK; region_id FK; hostname UK; total_vcpu, total_ram_gb, total_disk_gb; used_vcpu, used_ram_gb, used_disk_gb (CHECK used <= total); status ENUM(active,maintenance,retired)
instance_types	type_id PK; name UK; vcpu; ram_gb; disk_gb (all CHECK > 0)
pricing	price_id PK; type_id FK; price_per_hour DECIMAL(8,4) CHECK > 0; valid_from DATE; valid_to DATE NULL; no overlap per type (trigger)
images	image_id PK; name; os_family ENUM(linux,windows,bsd); version; min_disk_gb
instances	instance_id PK; org_id FK; host_id FK; type_id FK; image_id FK; name; status ENUM(pending,running,stopped,suspended,terminated); config JSON; auto_backup (generated from JSON, indexed); created_at; terminated_at. UK(org_id, name)
volumes	volume_id PK; org_id FK; instance_id FK NULL; size_gb CHECK > 0; vol_type ENUM(ssd,hdd); status ENUM(available,attached,deleted)
snapshots	snapshot_id PK; volume_id FK; size_gb; created_at
ip_addresses	ip_id PK; region_id FK; ip_address UK; org_id FK NULL; instance_id FK NULL UK; allocated_at NULL
domains	domain_id PK; org_id FK; name UK; registered_on; expires_on CHECK > registered_on
dns_records	record_id PK; domain_id FK; rtype ENUM(A,AAAA,CNAME,MX,TXT); name; value; ttl CHECK 60..86400; UK(domain_id, rtype, name, value)
usage_records	usage_id, recorded_at composite PK; instance_id; org_id; metric ENUM(cpu_hour,storage_gb,egress_gb); quantity DECIMAL(10,3) CHECK >= 0. PARTITION BY RANGE (month). No FKs (see 6.3)
invoices	invoice_id PK; org_id FK; period_start; period_end; subtotal, tax, credit_applied, total DECIMAL(12,2); status ENUM(draft,issued,paid,overdue,void); issued_at; due_date. UK(org_id, period_start)
invoice_items	item_id PK; invoice_id FK; instance_id FK NULL; description; quantity; unit_price; amount = quantity * unit_price (CHECK)
payments	payment_id PK; invoice_id FK; amount CHECK > 0; method ENUM(card,upi,netbanking,credit); txn_ref UK; paid_at
tickets	ticket_id PK; org_id FK; created_by FK; assigned_to FK NULL; subject; priority ENUM(low,normal,high,urgent); status ENUM(open,pending,resolved,closed); created_at; resolved_at
ticket_messages	msg_id PK; ticket_id FK; sender_id FK; body TEXT with FULLTEXT index; created_at
audit_log	audit_id PK; table_name; record_id; action ENUM(INSERT,UPDATE,DELETE); old_data JSON; new_data JSON; changed_by; changed_at
daily_usage_summary	(usage_date, org_id) composite PK; cpu_hours; storage_gb; egress_gb; cost (rolled up by event)
dim_org / dim_type / dim_date / fact_usage_daily	Star schema for OLAP reports: fact table keyed by the three dimensions with measures cpu_hours, cost

6. Database Engineering Rules

6.1 Naming and type conventions

- snake_case, singular column names, plural table names; surrogate integer PKs with AUTO_INCREMENT.
- Money is DECIMAL, never FLOAT. Timestamps in UTC. Booleans as BOOLEAN. Fixed value sets as ENUM or lookup tables.
- Every FK column is indexed; every table has created_at where relevant.

6.2 Foreign key action policy

Relationship	ON DELETE	Reason
instances -> organizations/hosts/types/images	RESTRICT	History and billing must not vanish when a parent is removed.
invoice_items, payments -> invoices	RESTRICT	Financial records are immutable.
org_members -> organizations/users	CASCADE	Membership has no meaning without either side.
dns_records -> domains	CASCADE	Records belong entirely to the domain.
snapshots -> volumes	RESTRICT	Do not silently lose backups.
volumes.instance_id, ip_addresses.instance_id	SET NULL	Resources outlive the instance (detach, release).
ticket_messages -> tickets	CASCADE	Messages belong to the ticket.

6.3 Integrity constraints to implement and demonstrate

Kind	Examples
PRIMARY KEY	All tables; composite keys on org_members and usage_records.
FOREIGN KEY	All relationships in Section 4 with the action policy above.
UNIQUE	users.email, hosts.hostname, ip_addresses.ip_address, payments.txn_ref, instances(org_id, name), invoices(org_id, period_start).
CHECK	used_vcpu <= total_vcpu, price_per_hour > 0, quantity >= 0, due_date >= issued_at, amount = quantity * unit_price, ttl range.
NOT NULL / DEFAULT	status defaults, created_at defaults, mandatory business fields.
Domain constraints	ENUM status machines, DECIMAL precision, VARCHAR lengths.
Trigger-enforced rules	Quota limits, pricing no-overlap, status transition rules, immutability of paid invoices, usage row validity.

> Design decision to defend in the viva: MySQL does not allow foreign keys on partitioned InnoDB tables. usage_records is partitioned for performance, so its integrity is enforced by the only insert path (stored procedure sp_record_usage plus a BEFORE INSERT trigger that verifies the instance exists) and verified by the consistency check in Section 9.3. Also, MySQL forbids CHECK constraints on columns used in CASCADE or SET NULL foreign keys, so those rules are written as triggers.

6.4 Normalization plan and proof outline

The report must show the process, not just the final schema. Start from one unnormalized table and normalize step by step.

```
UNF: Billing(org_name, org_email, tier_name, tier_limit, instance_name, type_name, vcpu, ram, host, region,
             usage_hour_list, invoice_no, invoice_total, payments_list)

1NF  : remove repeating groups (usage_hour_list, payments_list) -> rows in usage_records and payments.
2NF  : key (org, instance, invoice) has partial dependencies:
         org_name, tier_name -> depend on org only     => organizations
         type_name, vcpu, ram -> depend on type only   => instance_types
         host, region         -> depend on host only   => hosts, regions
3NF  : transitive dependencies removed:
         org -> tier_id -> tier_limit                  => account_tiers
         host -> region_id -> region_name              => regions
         invoice_total depends on items                => invoice_items (total derived, see below)
BCNF : for every table list the functional dependencies and show each determinant is a candidate key.
```

Deliberate denormalization (document and justify each)	Why it is acceptable	How consistency is kept
hosts.used_vcpu / used_ram_gb / used_disk_gb	Capacity check must be O(1) for the scheduler.	Maintained only by triggers; verified by sp_check_consistency.
invoices.subtotal / total	Immutable snapshot of what the customer was charged.	Written once by sp_generate_invoice; CHECK total = subtotal + tax - credit_applied.
organizations.credit_balance	Fast balance lookup.	Changed only inside transactional procedures; audit logged.
daily_usage_summary and fact tables	Analytics without scanning millions of rows.	Rebuilt by event; reconciliation query compares with raw usage.

7. Stored Programs

7.1 Stored procedures

Procedure	Purpose and logic
sp_launch_instance(org, type, image, region, name)	Transaction. Loads type specs, picks best-fit host with FOR UPDATE SKIP LOCKED, inserts instance, allocates IP. Rolls back on any failure.
sp_terminate_instance(instance)	Marks terminated, detaches volumes (SET NULL), releases IP; trigger frees host capacity.
sp_resize_instance(instance, new_type)	Checks the same host has room, else migrates through the scheduler; savepoint to undo on failure.
sp_record_usage(instance, metric, qty, ts)	Validates instance state and inserts one usage row (the only insert path into usage_records).
sp_generate_invoice(org, period_start, period_end)	Aggregates usage, prices with temporal lookup, adds tax, applies credit, creates invoice and items atomically.
sp_run_monthly_billing(period)	Cursor over active organizations calling sp_generate_invoice; per-org error handler logs failures and continues.
sp_apply_payment(invoice, amount, method, ref)	Inserts payment; trigger updates invoice status; reactivates suspended instances when no overdue invoice remains.
sp_take_snapshot(volume) / sp_restore_snapshot(snapshot)	Quota check, create snapshot or new volume in one transaction.
sp_rollup_daily_usage(date)	Idempotent INSERT ... ON DUPLICATE KEY UPDATE into daily_usage_summary and fact table.
sp_check_consistency()	Returns a result set of every integrity violation found (see Section 9.3).
sp_drop_old_partitions(months)	Drops expired monthly partitions of usage_records after archiving.

7.2 Stored functions

Function	Returns
fn_hourly_price(type_id, at_time)	Price valid at the given moment from the temporal pricing table.
fn_org_running_instances(org_id)	Number of running or stopped (billable) instances.
fn_org_quota_left(org_id)	Instances still allowed under the tier.
fn_invoice_balance(invoice_id)	Total minus sum of payments.
fn_host_utilization(host_id)	Weighted utilization percentage across CPU, RAM, disk.

7.3 Triggers

Trigger	Event	Behaviour
trg_instances_bi_quota	BEFORE INSERT instances	SIGNAL error when the organization reached its tier limit or is suspended.
trg_instances_ai_capacity	AFTER INSERT instances	Adds type vcpu/ram/disk to host used columns.
trg_instances_au_status	AFTER UPDATE instances	On terminate, subtracts capacity; validates allowed status transitions.
trg_hosts_bu_limit	BEFORE UPDATE hosts	Rejects used values above total (second line of defence besides CHECK).
trg_pricing_bi_overlap	BEFORE INSERT/UPDATE pricing	Rejects overlapping validity ranges for the same type.
trg_usage_bi_valid	BEFORE INSERT usage_records	Verifies instance exists and was running (replaces the missing FK).
trg_payments_ai_status	AFTER INSERT payments	Marks invoice paid when payments cover the total.
trg_invoices_bu_immutable	BEFORE UPDATE invoices	Blocks changes to amounts once issued; only status may move forward.

Trigger	Event	Behaviour
trg_invoices_bd_block	BEFORE DELETE invoices, payments, audit_log	Always SIGNAL: financial and audit records are never deleted.
trg_audit_* (organizations, instances, invoices, payments, users, hosts)	AFTER INSERT/UPDATE/DELETE	Write old and new row as JSON into audit_log.

7.4 Views

View	Use
v_host_capacity	Free and used CPU/RAM/disk per host and region, utilization percent.
v_running_instances	Instances joined with organization, type, host, region, IP.
v_org_monthly_cost	Cost per organization per month from usage and pricing.
v_overdue_invoices	Invoices past due with days overdue and aging bucket.
v_ticket_sla	Open tickets with hours waiting versus priority target.
v_revenue_by_region	Revenue aggregated by region and type for the admin dashboard.
v_my_instances (WITH CHECK OPTION)	Tenant-scoped, updatable view used by the customer database account.

7.5 Scheduled events (MySQL Event Scheduler)

Event	Schedule	Action
ev_hourly_metering	Every hour (every minute in demo mode)	Writes usage rows for running instances via sp_record_usage.
ev_nightly_rollup	Daily 01:00	sp_rollup_daily_usage for yesterday.
ev_monthly_invoicing	Monthly, day 1	sp_run_monthly_billing for the previous month.
ev_suspend_overdue	Daily 02:00	Marks invoices overdue, suspends instances after grace days.
ev_purge_usage	Monthly	sp_drop_old_partitions after archiving.

7.6 Reference code: scheduler procedure

```
DELIMITER $$
CREATE PROCEDURE sp_launch_instance(IN p_org INT, IN p_type INT, IN p_image INT,
                                    IN p_region INT, IN p_name VARCHAR(60))
BEGIN
  DECLARE v_host INT DEFAULT NULL;
  DECLARE v_cpu INT; DECLARE v_ram INT; DECLARE v_disk INT;
  DECLARE EXIT HANDLER FOR SQLEXCEPTION BEGIN ROLLBACK; RESIGNAL; END;

  SELECT vcpu, ram_gb, disk_gb INTO v_cpu, v_ram, v_disk FROM instance_types WHERE type_id = p_type;

  START TRANSACTION;
  SELECT host_id INTO v_host FROM hosts
   WHERE region_id = p_region AND status = 'active'
     AND total_vcpu - used_vcpu >= v_cpu AND total_ram_gb - used_ram_gb >= v_ram
     AND total_disk_gb - used_disk_gb >= v_disk
   ORDER BY (total_vcpu - used_vcpu) ASC          -- best fit: fill fuller hosts first
   LIMIT 1 FOR UPDATE SKIP LOCKED;                -- concurrent launches never pick the same host row

  IF v_host IS NULL THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'No capacity available in this region';
  END IF;

  INSERT INTO instances(org_id, host_id, type_id, image_id, name, status)
  VALUES (p_org, v_host, p_type, p_image, p_name, 'running');   -- triggers: quota check, capacity, audit
  CALL sp_allocate_ip(LAST_INSERT_ID(), p_region);
  COMMIT;
END$$
DELIMITER ;
```

7.7 Reference code: audit trigger and invoice item pricing

```
CREATE TRIGGER trg_audit_instances_au AFTER UPDATE ON instances FOR EACH ROW
  INSERT INTO audit_log(table_name, record_id, action, old_data, new_data, changed_by)
  VALUES ('instances', NEW.instance_id, 'UPDATE',
          JSON_OBJECT('status', OLD.status, 'type_id', OLD.type_id, 'host_id', OLD.host_id),
          JSON_OBJECT('status', NEW.status, 'type_id', NEW.type_id, 'host_id', NEW.host_id),
          COALESCE(@app_user_id, 0));        -- backend runs SET @app_user_id = ? per request

-- inside sp_generate_invoice: price each hour with the price that was valid at that hour
INSERT INTO invoice_items(invoice_id, instance_id, description, quantity, unit_price, amount)
SELECT v_invoice, i.instance_id, CONCAT('Compute - ', i.name), SUM(u.quantity), p.price_per_hour,
       ROUND(SUM(u.quantity) * p.price_per_hour, 2)
FROM usage_records u
JOIN instances i ON i.instance_id = u.instance_id
JOIN pricing p   ON p.type_id = i.type_id
                AND DATE(u.recorded_at) >= p.valid_from
                AND (p.valid_to IS NULL OR DATE(u.recorded_at) <= p.valid_to)
WHERE u.org_id = p_org AND u.metric = 'cpu_hour'
  AND u.recorded_at >= p_start AND u.recorded_at < DATE_ADD(p_end, INTERVAL 1 DAY)
GROUP BY i.instance_id, i.name, p.price_id, p.price_per_hour;
```

8. Query Optimization and Performance Engineering

This section is the performance analysis the guidelines require in the final report. Every claim must be backed by an EXPLAIN ANALYZE screenshot and a timing table.

8.1 Method (repeat for every benchmark query)

1. Load benchmark data (Section 2.4) using the generator script; run ANALYZE TABLE on all tables.
2. Run the query with only primary keys. Record EXPLAIN ANALYZE: access type, rows examined, actual time, loops.
3. Add the proposed index. Run ANALYZE TABLE again. Re-run EXPLAIN ANALYZE.
4. Run the query 100 times through the Node benchmark script; record average, p95 and rows examined.
5. Fill the comparison table and write one paragraph explaining why the plan changed.

8.2 Benchmark queries and planned indexes

ID	Query	Index or technique	Expected plan change
Q1	Usage of one instance in a date range	Composite (instance_id, recorded_at) covering metric, quantity	ALL to range scan, index only
Q2	Running instances of an organization	Composite (org_id, status)	ALL to ref
Q3	Overdue invoices list	Composite (status, due_date)	ALL to range
Q4	Invoice with items and payments (3-table join)	FK indexes on invoice_items(invoice_id), payments(invoice_id)	Nested loop with ref lookups
Q5	Search tickets by words	FULLTEXT on ticket_messages(body) with MATCH AGAINST	LIKE with leading % (full scan) to fulltext
Q6	Usage for one month across all orgs	RANGE partitioning by month (partition pruning)	Only one partition read
Q7	Instances with auto backup enabled	Generated column from JSON config plus index	JSON scan to index ref
Q8	Audit history of one record	Composite (table_name, record_id, changed_at)	ALL to ref, no filesort
Q9	Top 10 spenders this quarter	Summary table plus index versus raw scan	Raw 1M rows versus summary 50k rows
Q10	Hosts that can fit a given type	Composite on (region_id, status)	Smaller candidate set

8.3 Techniques to demonstrate

- **Index types:** single column, composite (left-most prefix rule shown with a query that cannot use it), covering index, unique index, FULLTEXT, index on generated column, descending index, invisible index to test the effect of dropping an index safely.
- **Partitioning:** RANGE COLUMNS by month on usage_records; show partition pruning in EXPLAIN (partitions column) and instant retirement of old data with DROP PARTITION.
- **Query rewriting:** correlated subquery to JOIN, IN to EXISTS, OFFSET pagination to keyset pagination, SELECT * to explicit columns, OR to UNION where useful.
- **Anti-patterns to prove wrong:** function on an indexed column (WHERE DATE(recorded_at) = ...), leading wildcard LIKE, implicit type conversion, SELECT * with large TEXT columns, N+1 queries from the API.
- **Tools:** EXPLAIN, EXPLAIN ANALYZE, EXPLAIN FORMAT=TREE, slow query log with long_query_time = 0.1, SHOW INDEX, performance_schema or sys schema (sys.statement_analysis, sys.schema_unused_indexes), mysqlslap for concurrent load.
- **Write-cost trade-off:** measure insert throughput of usage_records with 0, 2 and 5 indexes to show that indexes slow writes.

8.4 Results table (to be filled with real numbers)

Query	Time before (ms)	Time after (ms)	Rows examined before	Rows examined after	Plan before	Plan after	Speedup
Q1							
Q2							
Q3							
Q5							
Q6							
Q7							

8.5 Reference snippets

```
-- partitioned metering table (no FKs allowed on partitioned InnoDB tables)
CREATE TABLE usage_records (
  usage_id BIGINT AUTO_INCREMENT, recorded_at DATETIME NOT NULL,
  instance_id INT NOT NULL, org_id INT NOT NULL,
  metric ENUM('cpu_hour','storage_gb','egress_gb') NOT NULL,
  quantity DECIMAL(10,3) NOT NULL CHECK (quantity >= 0),
  PRIMARY KEY (usage_id, recorded_at)
) PARTITION BY RANGE COLUMNS (recorded_at) (
  PARTITION p2026_09 VALUES LESS THAN ('2026-10-01'),
  PARTITION p2026_10 VALUES LESS THAN ('2026-11-01'),
  PARTITION pmax     VALUES LESS THAN (MAXVALUE));

CREATE INDEX idx_usage_inst_time ON usage_records(instance_id, recorded_at, metric, quantity);  -- covering

EXPLAIN ANALYZE
SELECT metric, SUM(quantity) FROM usage_records
WHERE instance_id = 4211 AND recorded_at >= '2026-10-01' AND recorded_at < '2026-11-01'
GROUP BY metric;
```

9. Transactions, Concurrency, Audit and Integrity

9.1 Concurrency demonstrations (two MySQL sessions side by side)

Demo	What to show	Fix shown
Lost update	Two sessions read credit_balance, both subtract, one update is lost.	SELECT ... FOR UPDATE or atomic UPDATE with condition.
Overbooking	Two launches pick the same last slot on a host.	Row lock plus SKIP LOCKED inside sp_launch_instance.
Isolation levels	Non-repeatable read under READ COMMITTED versus stable read under REPEATABLE READ; phantom rows with range locks.	Choose the level per operation and explain why.
Deadlock	Session A locks host then instance, session B the reverse; InnoDB kills one.	Consistent lock ordering; retry on error 1213 in the API.
Atomic billing	Failure halfway through invoice creation.	Transaction plus SAVEPOINT; nothing half-written.
Durability	Kill the MySQL process after COMMIT.	Redo log recovery; data intact.

9.2 Audit design

- audit_log stores table, record id, action, old and new row as JSON, user and timestamp. Written only by triggers, never by the application.
- The backend sets a session variable (SET @app_user_id) at the start of every request so triggers can attribute changes to the real user.
- Audit viewer in the admin UI: filter by table, record, user and date; show a before/after diff.
- Immutability: BEFORE DELETE triggers on invoices, payments and audit_log; the application database user has no DELETE privilege on them.
- Soft delete (status = terminated or deleted_at) for business entities; hard delete is for test data only.

9.3 Consistency check procedure: sp_check_consistency

Returns one row per violation with a check name, record id and message. Run it from the admin dashboard and after every bulk load.

#	Check
1	hosts.used_* equals the sum of specs of non-terminated instances on that host.
2	No host has used greater than total on any resource.
3	invoices.total equals sum of invoice_items.amount plus tax minus credit.
4	Invoices marked paid have payments summing to at least the total; unpaid invoices have less.
5	No usage rows for instances that do not exist (replacement for the missing FK).
6	No instance running on a host that is retired or in maintenance.
7	Every running instance has exactly one IP; no IP is assigned to a terminated instance.
8	No organization has more active instances than its tier allows.
9	Pricing ranges do not overlap and have no gaps for active types.
10	credit_balance is never negative; daily summary equals raw usage totals.

9.4 Security and access control

- Passwords hashed with bcrypt; API keys stored as hashes; JWT with expiry.
- Parameterized queries only (mysql2 placeholders) to prevent SQL injection; show one attempted injection failing in the test report.
- MySQL roles: db_admin, db_support, db_billing, db_customer with GRANT on views and EXECUTE on procedures only.
- Tenant isolation: customer queries go through views filtered by organization.

- Backups: mysqldump with --routines --triggers --events nightly; binary logging enabled; demonstrate point-in-time recovery after an accidental DELETE.

10. Advanced Querying Catalogue (reports)

Each report is implemented as a view or a parameterized query behind an API endpoint and appears in the Reports screens. The report must list these with the concept used.

#	Report	Concepts
R1	Monthly revenue by region and instance type with subtotals and grand total	JOIN (5 tables), GROUP BY WITH ROLLUP
R2	Top 10 customers by spend with rank and share of total	Window RANK(), SUM() OVER ()
R3	Month-over-month revenue growth percent	Window LAG()
R4	Running cost within the month per organization	Window SUM() OVER (ORDER BY ...)
R5	Customer segmentation into quartiles by spend	NTILE(4)
R6	Total cost of an organization including all child organizations	Recursive CTE on parent_org_id
R7	Resource dependency tree: instance, volumes, snapshots, IP	Multi-table JOIN, UNION ALL
R8	Hosts above 85 percent utilization, by region	Aggregate, HAVING, view
R9	Customers who never raised a ticket but spend above average	Subquery, NOT EXISTS
R10	Overdue aging buckets (0-30, 31-60, 61-90, 90+)	CASE, GROUP BY, DATEDIFF
R11	Instances launched per hour of day (peak load)	GROUP BY HOUR(), aggregate
R12	Instances by image OS and tags stored in JSON config	JSON_TABLE, JSON_EXTRACT
R13	Price paid under old versus new pricing for the same usage (as of date)	Temporal join on valid_from / valid_to
R14	Ticket full-text search with relevance score	MATCH ... AGAINST
R15	Average ticket resolution time per agent	JOIN, AVG, TIMESTAMPDIFF
R16	Host capacity forecast: days until region full at current growth	CTE, window functions, arithmetic
R17	Pivot of monthly cost per resource type (columns per metric)	Conditional aggregation
R18	Organizations with usage but no invoice (billing gap detection)	LEFT JOIN ... IS NULL
R19	Cumulative distribution of invoice size (percentiles)	PERCENT_RANK / CUME_DIST
R20	Audit trail of one invoice with who changed what	JSON_EXTRACT, JOIN users

11. Application Architecture and Implementation Plan

11.1 Architecture

Layer	Technology	Responsibility
Presentation	React (Vite), React Router, Recharts	Role-based screens, forms, charts, live dashboards (polling every few seconds).
API	Node.js, Express, mysql2 pool, JWT, bcryptjs	Authentication, role checks, calls stored procedures, input validation, error mapping of SQLSTATE 45000 to user messages.
Database	MySQL 8 InnoDB	All business rules: procedures, triggers, events, views, constraints.
Simulator	Node script	Generates users, instances and usage in bulk; live mode creates fake events every few seconds for demos.
Deployment	Cloud MySQL, Render or Railway, Vercel	Public demo. Verify current free-tier limits before choosing providers.

11.2 API endpoints (REST)

Area	Endpoints
Auth	POST /auth/login, POST /auth/register, GET /auth/me
Organizations	GET/POST /orgs, GET /orgs/:id/members, POST /orgs/:id/members, GET /orgs/:id/cost-tree
Catalog	GET /instance-types, GET /images, GET /pricing, POST /pricing (admin)
Infrastructure	GET/POST /regions, GET/POST/PUT /hosts, GET /hosts/capacity
Compute	GET /instances, POST /instances (CALL sp_launch_instance), POST /instances/:id/{start,stop,resize,terminate}
Storage and network	GET/POST /volumes, POST /volumes/:id/attach, POST /volumes/:id/snapshot, GET/POST /ips, GET/POST /domains, GET/POST /domains/:id/records
Billing	GET /invoices, GET /invoices/:id, POST /invoices/generate, POST /invoices/:id/pay, GET /usage
Support	GET/POST /tickets, POST /tickets/:id/messages, PUT /tickets/:id/status, GET /tickets/search?q=
Admin and analytics	GET /reports/R1 ... R20, GET /audit, POST /admin/consistency-check, GET /admin/slow-queries, POST /admin/simulate

11.3 Frontend screens

Role	Screens
Customer	Dashboard (running instances, month-to-date cost chart); Launch wizard (region, type with live price, image, review); Instances list with start/stop/resize/terminate; Volumes and snapshots; IPs; Domains and DNS editor; Invoices and invoice detail with Pay; Usage explorer; Tickets; Team and API keys.
Billing Manager	Invoices for all organizations, generate invoice, apply credit, overdue aging report, revenue charts.
Support Agent	Ticket queue with SLA colours, ticket thread, customer overview.
Super Admin	Capacity dashboard (hosts, heat map of utilization), pricing editor with history, region and host management, audit viewer with diff, consistency checker, performance lab page (runs benchmark queries and shows timings and EXPLAIN output), reports R1 to R20, simulator controls.

Highlight feature for the demo: the Performance Lab page. It runs the same query with and without an index (using an invisible index toggle) and shows EXPLAIN ANALYZE output and timings side by side. Examiners see the optimization work live.

11.4 Folder structure

```
nimbushost/
  database/   01_schema.sql  02_functions.sql  03_procedures.sql  04_triggers.sql  05_views.sql
              06_events.sql  07_indexes.sql  08_roles.sql  09_seed_demo.sql  10_reports.sql
              11_consistency.sql  12_benchmark_queries.sql
  simulator/  generate_data.js  live_usage.js
  backend/    src/{server.js, db.js, middleware/, routes/}   .env.example
  frontend/   src/{pages/, components/, api.js, App.jsx}
  docs/       ER_diagram.png  relational_schema.md  data_dictionary.md  normalization.md
              test_cases.md  performance_analysis.md  screenshots/
  README.md
```

12. Documentation and Deliverables Checklist

Mapped one-to-one to Section 6 of the course guidelines. Tick each item and attach evidence.

Deliverable (guideline)	Contents to include	Owner	Done
Executive Summary	Problem statement, objectives, scope (Section 1 of this document, polished)	M1	[ ]
Final presentation deck	12 to 15 slides: problem, architecture, ER diagram, key procedures, concurrency demo, performance results, live demo, conclusion	All	[ ]
ER diagram	All 24 entities, attributes, PK/FK, cardinality, crow-foot notation, high resolution	M1	[ ]
Relational schema	Every table with keys and FK arrows	M1	[ ]
Data dictionary	Table, column, type, constraint, description for every column	M1	[ ]
Normalization proof	UNF to 1NF to 2NF to 3NF to BCNF with FDs per table; justified denormalization	M1	[ ]
Advanced DBMS topics	Procedures, functions, triggers, views, events, cursors, transactions, locking, isolation, partitioning, indexing, JSON, window functions, CTE, roles, replication or backup	M2	[ ]
Core SQL scripts	All files in database/ runnable in order on an empty server	M2	[ ]
UI screenshots	High resolution, every role, every module, error states	M4	[ ]
Test cases and results	Section 13 table with actual results and screenshots	M3	[ ]
Performance analysis	Section 8 tables and EXPLAIN ANALYZE screenshots with explanation	M2	[ ]
Conclusion and evaluation	What was achieved, limitations, future work (sharding, read replicas, real metering)	All	[ ]

13. Verification: Test Plan

Execute every case, record Actual result and a screenshot. Types: SQL (S), API (A), UI (U), concurrency (C), security (X).

ID	Type	Scenario	Expected result
T01	S	Launch instance with free capacity	Instance running, host used values increase, audit row created
T02	S	Launch when no host has capacity	Error "No capacity", nothing inserted, transaction rolled back
T03	S	Launch beyond tier instance limit	Quota trigger rejects the insert
T04	S	Launch for suspended organization	Rejected with clear message
T05	S	Terminate instance	Status terminated, host capacity freed, IP released, volumes detached
T06	S	Insert host with used_vcpu above total	CHECK constraint violation
T07	S	Delete an organization that has instances	FK RESTRICT error
T08	S	Insert duplicate email, duplicate IP, duplicate txn_ref	UNIQUE violations
T09	S	Insert overlapping price range for one type	Pricing trigger rejects
T10	S	Generate invoice for a month with price change	Items use the price valid on each date
T11	S	Generate the same invoice twice	Second call rejected by unique (org_id, period_start)
T12	S	Pay invoice in two parts	Status becomes paid only when sum covers total
T13	S	UPDATE amount on an issued invoice	Immutability trigger rejects
T14	S	DELETE from audit_log and payments	Always rejected
T15	S	Overdue event runs	Invoices marked overdue, instances suspended; payment reactivates
T16	S	Cursor billing with one failing organization	Other organizations invoiced, failure logged
T17	S	Run sp_check_consistency on clean data	Zero rows returned
T18	S	Manually corrupt hosts.used_vcpu, run check	Violation reported with host id
T19	C	Two sessions launch for the last slot	Exactly one succeeds
T20	C	Forced deadlock	One session fails with 1213; retry succeeds
T21	C	Concurrent payments on one invoice	No lost update, correct final state
T22	A	Request admin endpoint with customer token	403
T23	A	Request without or with expired token	401
T24	X	SQL injection string in search box	Treated as data, no effect
T25	U	Launch wizard happy path and error path	Success toast; readable error message from the database
T26	U	Tenant isolation: customer A opens customer B instance URL	Not found or forbidden
T27	S	Restore from mysqldump plus binlog to a point in time	Deleted rows recovered
T28	S	Partition pruning on month query	EXPLAIN shows one partition

Performance and conclusion section outline

- Dataset size and hardware description; MySQL version and key settings (innodb_buffer_pool_size).
- Results table from Section 8.4 with EXPLAIN ANALYZE screenshots before and after each index.
- Concurrency test: throughput of mysqlslap or the Node script with 1, 10, 50 clients.
- Write-cost experiment with different numbers of indexes.
- Conclusion: objectives met, limitations (simulated infrastructure, single node), future work (read replica, sharding by region, real metering agents).

14. Team Roles and Timeline

Every member must be able to explain and defend any part. Roles below assume four members; with three, merge Members 3 and 4 duties.

Member	Role	Owns (and must defend)
Member 1	Data architect	ER diagram, schema DDL, constraints, normalization proof, data dictionary, seed and generator design.
Member 2	Database programmer and performance lead	Functions, procedures, triggers, views, events, indexes, partitioning, EXPLAIN ANALYZE benchmarks, concurrency demos.
Member 3	Backend and security	Express API, JWT and roles, MySQL roles and GRANTs, audit viewer, consistency checks, simulator scripts, deployment.
Member 4	Frontend and QA	React screens, charts, Performance Lab page, test execution and evidence, screenshots, report and slides assembly.

Timeline (adjust to the submission date given by the department)

Week	Goal	Output
1	Specification and design	Roles, workflows, ER diagram v1, table list, repo created, MySQL running for everyone.
2	Schema and integrity	Final DDL with constraints, normalization proof, data dictionary draft, seed data.
3	Stored programs	Functions, procedures, triggers, views, events; unit tests in SQL.
4	Backend plus data generator	API with auth and role checks, simulator producing 1M usage rows.
5	Frontend and integration	All role screens, dashboards, error handling, end-to-end demo path.
6	Optimization, testing, documentation	Benchmarks, concurrency demos, test results, report, slides, rehearsal.

Risks and mitigations

Risk	Mitigation
Scope too large for the time	Build in layers: Layer 1 core schema and billing; Layer 2 scheduler and audit; Layer 3 partitioning, events, analytics. Layer 1 and 2 alone already exceed the guideline.
MySQL feature mismatch across laptops	Everyone uses MySQL 8.0.30 or later; share one SQL script set; use a common cloud database for integration.
Benchmark numbers look unimpressive	Use enough rows (1M or more), cold cache restarts, and queries where the optimizer must choose; report honestly, including cases where an index did not help.
Events and triggers hard to debug	Write each as a script with a test block; keep a debug table for logging messages.
Demo failure	Pre-recorded fallback video, local copy of the database, scripted demo order.

15. Viva Preparation: Questions to Practice

Question	Points to cover
Why this schema and why 3NF/BCNF?	Walk the normalization steps; show an anomaly in the unnormalized version.
Where did you denormalize and why?	Host used counters, invoice totals, summary tables; how triggers and checks keep them correct.
How do you prevent two users taking the last host slot?	Transaction, SELECT FOR UPDATE SKIP LOCKED, InnoDB row locks, demo T19.
Explain ACID with an example from your project.	Invoice generation or launch procedure; rollback demo; redo log for durability.
Which isolation level do you use and why?	REPEATABLE READ default, READ COMMITTED for reports; phantom and non-repeatable read examples.
What is a deadlock and how do you handle it?	Lock ordering, error 1213, retry in the API.
Why is usage_records partitioned and what is the catch?	Pruning and fast purge; no foreign keys on partitioned tables, so a procedure and trigger enforce integrity.
How did the index change the plan?	Show EXPLAIN ANALYZE: access type, rows examined, time; left-most prefix rule.
When is an index harmful?	Write cost experiment; low-selectivity columns; unused indexes found with sys schema.
Trigger versus procedure versus application code?	Rules that must hold for every writer go into triggers or constraints; workflows into procedures.
How is the audit trail tamper-resistant?	Trigger-written, delete-blocked, no DELETE grant, consistency checks.
How do you handle price changes?	Temporal pricing with valid_from and valid_to; as-of join in invoice generation.
How do you secure the system?	Hashing, JWT, role checks, MySQL roles, views for tenants, parameterized queries.
How would this scale beyond one server?	Read replicas for reports, sharding by region, archiving old partitions, connection pooling.
What did you do personally?	Each member prepares a two-minute walkthrough of their own modules.

16. Final Submission Checklist

- All SQL scripts run in order on a fresh MySQL 8 server without errors (test on a second machine).
- Application starts with two commands per tier; README has setup steps and demo credentials for each role.
- ER diagram, schema, dictionary, normalization proof are consistent with the real database (regenerate diagram from the final DDL).
- Every guideline item in Section 12 ticked with evidence.
- Test table complete with actual results and screenshots; performance tables filled with real measurements.
- Slides rehearsed; fallback demo recording saved; each member can explain every module.
- Report compiled as a single PDF with page numbers and table of contents; code zipped separately.